

Semana 14 Clase 1 Inteligencia Artificial

*Resumen de la clase impartida el día martes 26 de mayo

Emmanuel José Matamoros Jiménez
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
emmajose1703@estudiantec.cr

Profesor: Steven Andrey Pacheco Portuquez
Curso: IC6200 Inteligencia Artificial

Abstract—Este documento resume los principales conceptos abordados durante la clase relacionados con embeddings, búsqueda semántica, sistemas RAG (Retrieval Augmented Generation) y la construcción de agentes basados en modelos de lenguaje (LLMs). Además, se analizan los componentes fundamentales de un agente inteligente, los criterios para la selección de modelos de lenguaje y las diferencias entre modelos propietarios y modelos abiertos utilizados en sistemas modernos de inteligencia artificial.

I. INICIO DE LA CLASE

A. Quiz 6

La sesión inició con la realización del último quiz del semestre, el cual abordó conceptos relacionados con redes neuronales convolucionales, arquitecturas de segmentación y modelos de aprendizaje no supervisado. A continuación las respuestas de dicho quiz.

1) ¿Qué es una capa de convolucion y una capa de pooling?:

Una capa de convolución es un componente fundamental de las Redes Neuronales Convolucionales (CNN), encargado de extraer características de una imagen mediante la aplicación de filtros o *kernels*. Estos filtros permiten detectar patrones como bordes, texturas y formas complejas, generando mapas de características que serán utilizados por las capas posteriores de la red.

Por otra parte, una capa de *pooling* tiene como objetivo reducir la dimensionalidad espacial de los mapas de características obtenidos durante la convolución. El método más utilizado es el *Max Pooling*, el cual conserva el valor máximo dentro de una región determinada, reduciendo el costo computacional y ayudando a disminuir el riesgo de sobreajuste.

2) Describa parte por parte un modulo de Inception:

Se repasó la composición de un módulo Inception, utilizado en arquitecturas como GoogLeNet. Este módulo permite procesar información a diferentes escalas de manera simultánea mediante cuatro ramas paralelas:

- Convolución 1×1 .
- Convolución 1×1 seguida de una convolución 3×3 .
- Convolución 1×1 seguida de una convolución 5×5 .

- Max Pooling seguido de una convolución 1×1 .

Las salidas de estas ramas se concatenan para formar una representación más rica de las características de entrada.

3) Describa como puede usarse un autoencoder para detección de anomalías:

También se revisó el uso de autoencoders para la detección de anomalías. Estos modelos se entrenan utilizando únicamente datos considerados normales, aprendiendo a reconstruirlos con un error mínimo.

Durante la fase de inferencia, si un dato presenta características significativamente diferentes a las observadas durante el entrenamiento, el error de reconstrucción aumenta considerablemente. Este incremento puede utilizarse como indicador para identificar anomalías, siendo una técnica ampliamente utilizada en detección de fraudes, monitoreo industrial y ciberseguridad.

4) Describa la composicion de una arquitectura UNet:

Finalmente, se repasó la estructura de la arquitectura U-Net, ampliamente utilizada en tareas de segmentación de imágenes. La arquitectura está compuesta por tres partes principales:

- **Encoder:** encargado de extraer características mediante convoluciones y operaciones de pooling.
- **Bottleneck:** representación comprimida de la información aprendida.
- **Decoder:** reconstruye progresivamente la resolución espacial de la imagen.

Una característica distintiva de U-Net son las *skip connections*, conexiones directas entre el encoder y el decoder que permiten preservar información espacial importante y mejorar la precisión de la segmentación.

B. Información para las siguientes evaluaciones

Posteriormente se discutió la planificación de las actividades restantes del curso:

- Semana 14: asignación de la siguiente tarea.
- Semana 15: asignación del proyecto final.
- Semana 18: realización del examen final y entrega del proyecto.

Además, se indicó que la próxima tarea estará orientada al uso práctico de modelos de lenguaje y herramientas relacionadas con agentes inteligentes, incluyendo tecnologías como LangFlow, LangSmith, MCP Servers y Agents.

C. Operaciones sobre embeddings

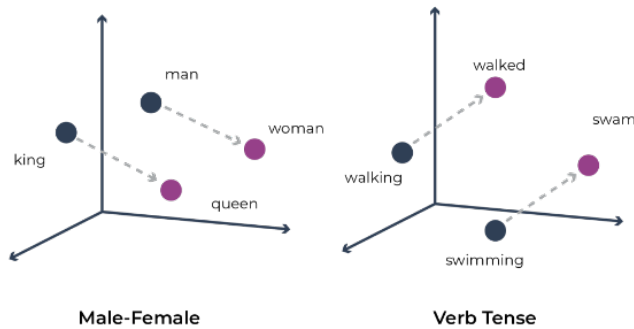


Fig. 1. Ejemplo del proceso de Embedding [1]

Una vez representados como vectores, es posible realizar diferentes operaciones matemáticas sobre los embeddings.

Uno de los ejemplos más conocidos consiste en realizar operaciones algebraicas entre conceptos:

$$\text{Rey} - \text{Hombre} + \text{Mujer} \approx \text{Reina}$$

Este comportamiento demuestra que los embeddings son capaces de capturar relaciones semánticas complejas presentes en el lenguaje natural.

D. Métricas de similitud

Para comparar embeddings se utilizan diferentes métricas matemáticas.

1) *Distancia Euclidiana*: La distancia euclidiana mide qué tan alejados se encuentran dos puntos dentro del espacio vectorial.

Una distancia pequeña indica una alta similitud entre los conceptos representados.

2) *Similitud del Coseno*: La similitud del coseno mide el ángulo entre dos vectores.

Mientras más pequeño sea el ángulo entre ellos, más similares serán sus significados.

Esta métrica es ampliamente utilizada en modelos de lenguaje debido a que se enfoca en la dirección de los vectores y no en su magnitud.

E. Embeddings de oraciones

Los embeddings no se limitan únicamente a palabras individuales. También es posible generar representaciones vectoriales de frases completas.

Estos *sentence embeddings* permiten comparar ideas, preguntas y documentos considerando su significado global, incluso cuando utilizan palabras diferentes.

Por ejemplo, una pregunta sobre la población de Berlín tendrá una representación mucho más cercana a una respuesta relacionada con dicha ciudad que a una oración referente a eventos deportivos o capitales de otros países.

II. LIMITACIONES DE LOS MODELOS DE LENGUAJE

Los modelos de lenguaje tradicionales poseen una limitación importante: su conocimiento es estático.

Una vez entrenado el modelo, la información que contiene queda congelada hasta que se realiza un nuevo proceso de entrenamiento. Esto dificulta responder preguntas relacionadas con información reciente o datos específicos de una organización.

Ante esta situación surge la necesidad de incorporar mecanismos que permitan acceder a conocimiento externo sin necesidad de reentrenar el modelo.

III. RETRIEVAL AUGMENTED GENERATION (RAG)

Una de las estrategias más utilizadas para resolver esta limitación es Retrieval Augmented Generation (RAG).

El objetivo de RAG consiste en complementar el conocimiento interno del modelo mediante información obtenida desde fuentes externas.

A. Proceso de construcción

El proceso inicia con una colección de documentos que contienen la información que se desea consultar.

Estos documentos son divididos en fragmentos más pequeños mediante una técnica conocida como *chunking*.

Cada fragmento es transformado posteriormente en un embedding utilizando un modelo especializado.

Finalmente, los vectores generados son almacenados dentro de una base de datos vectorial.

B. Proceso de consulta

Cuando el usuario realiza una pregunta:

- 1) La consulta es convertida en un embedding.
- 2) Se compara con todos los embeddings almacenados.
- 3) Se recuperan los fragmentos más similares.
- 4) Los fragmentos recuperados se agregan al prompt.
- 5) El modelo genera una respuesta utilizando dicha información adicional.

Este proceso recibe el nombre de *augmentación*, ya que aumenta el contexto disponible para el modelo durante la generación de la respuesta.

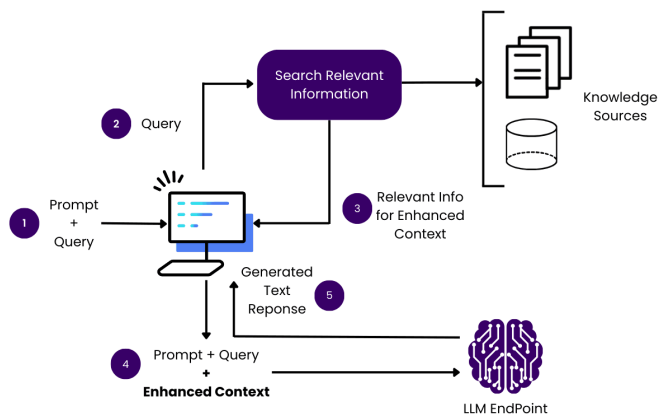


Fig. 2. Proceso RAG [2]

C. Aplicaciones de RAG

Entre los principales casos de uso se encuentran:

- Asistentes empresariales.
- Sistemas de soporte al cliente.
- Plataformas de investigación documental.
- Motores de búsqueda semántica.
- Sistemas de consulta sobre documentación interna.

IV. INTRODUCCIÓN A LOS AGENTES INTELIGENTES

La parte principal de la clase estuvo dedicada al estudio de agentes basados en modelos de lenguaje.

Mientras que un chatbot tradicional se limita a responder preguntas, un agente tiene la capacidad de razonar, planificar y ejecutar acciones.

Por esta razón puede considerarse una evolución natural de los asistentes conversacionales tradicionales.

Un agente combina múltiples componentes que trabajan conjuntamente para resolver tareas complejas de forma autónoma.

V. COMPONENTES FUNDAMENTALES DE UN AGENTE

A. Perfil

El perfil define la identidad del agente.

Este componente establece:

- Rol del agente.
- Personalidad.
- Objetivos.
- Restricciones de comportamiento.

El perfil permite adaptar el comportamiento del agente según el contexto de uso.

B. Memoria

La memoria permite mantener coherencia entre interacciones.

Existen dos tipos principales:

1) *Memoria de corto plazo:* Corresponde a la ventana de contexto del modelo.

Permite recordar información presente dentro de la conversación actual.

2) *Memoria de largo plazo:* Utiliza fuentes externas para almacenar información persistente.

Permite recordar preferencias del usuario o información relevante obtenida en sesiones anteriores.

C. Planificación

La planificación consiste en dividir tareas complejas en pasos más simples.

Este componente determina la secuencia de acciones que debe ejecutar el agente para alcanzar un objetivo determinado.

Entre las técnicas utilizadas destacan:

- Chain of Thought (CoT).
- Tree of Thoughts (ToT).
- Mecanismos de retroalimentación iterativa.

D. Acción

La acción representa la capacidad del agente para interactuar con su entorno.

Las acciones pueden ejecutarse utilizando:

- Sistemas RAG.
- Herramientas de búsqueda web.
- APIs externas.
- Servicios empresariales.
- Sistemas de bases de datos.

Esta capacidad permite que el agente vaya más allá de la generación de texto y produzca resultados ejecutables.

VI. SELECCIÓN DE MODELOS DE LENGUAJE

Uno de los retos más importantes en la construcción de agentes consiste en seleccionar el modelo de lenguaje adecuado.

No existe un único modelo ideal para todas las aplicaciones.

La selección depende de diversos factores técnicos y operativos.

A. Factores de evaluación

Entre los criterios más importantes se encuentran:

- Precisión requerida.
- Tiempo de respuesta.
- Costo computacional.
- Privacidad de los datos.
- Tamaño de la ventana de contexto.
- Capacidad de razonamiento.
- Soporte para herramientas externas.

Frecuentemente existe una relación de compromiso entre rendimiento, costo y latencia.

VII. EJECUCIÓN LOCAL DE MODELOS

La clase también abordó la posibilidad de ejecutar modelos de lenguaje localmente.

Herramientas como Ollama permiten descargar y ejecutar modelos directamente en equipos locales sin necesidad de utilizar APIs externas.

Entre las principales ventajas se encuentran:

- Mayor privacidad.
- Menor dependencia de proveedores externos.

- Control total de la infraestructura.
- Facilidad para experimentación y desarrollo.

Algunos modelos compatibles incluyen:

- Llama.
- Qwen.
- DeepSeek.
- Gemma.
- Mistral.

VIII. MODELOS PROPIETARIOS Y MODELOS ABIERTOS

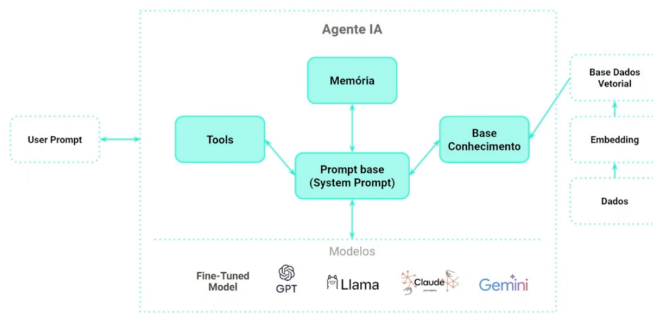


Fig. 3. Arquitectura de un Agente de IA [3]

Finalmente se discutieron los dos enfoques predominantes para el uso de modelos de lenguaje dentro de agentes inteligentes.

A. Modelos propietarios

Los modelos propietarios son administrados por empresas que controlan completamente su infraestructura y desarrollo.

Entre sus ventajas destacan:

- Facilidad de integración.
- Infraestructura administrada.
- Buen rendimiento general.
- Soporte avanzado para herramientas.

Sus principales limitaciones son:

- Dependencia del proveedor.
- Costos asociados al uso.
- Menor nivel de control sobre el modelo.

B. Modelos abiertos

Los modelos abiertos permiten un mayor grado de control y personalización.

Sus ventajas incluyen:

- Mayor privacidad.
- Personalización avanzada.
- Posibilidad de ejecución local.
- Control completo del entorno.

Sin embargo, presentan algunos desafíos:

- Mayor complejidad operativa.
- Necesidad de infraestructura propia.
- Responsabilidad sobre configuración y monitoreo.

IX. CONCLUSIONES

Durante esta sesión se estudió la evolución de los sistemas basados en modelos de lenguaje, desde los embeddings y la búsqueda semántica hasta la construcción de agentes inteligentes capaces de razonar y ejecutar acciones. Asimismo, se analizó cómo las arquitecturas RAG permiten complementar el conocimiento estático de los modelos mediante información externa, mejorando significativamente la calidad de las respuestas generadas. Finalmente, se revisaron los componentes fundamentales de un agente moderno y los criterios que deben considerarse para seleccionar adecuadamente un modelo de lenguaje según los requerimientos técnicos, económicos y operativos de cada aplicación.

X. REFERENCIAS

- [1] "Embeddings," Arize AI, Apr. 12, 2023. <https://arize.com/glossary/embeddings/> (accessed Jun. 03, 2026).
- [2] O. AI, "Understanding RAG: 6 Steps of Retrieval Augmented Generation (RAG)," Obot AI, May 21, 2024. <https://obot.ai/resources/learning-center/retrieval-augmented-generation/> (accessed Jun. 03, 2026).
- [3] "Todo lo que necesitas saber sobre los agentes de IA: guía definitiva," No-Code Start-Up, May 2024. <https://nocodestartup.io/es/guia-definitiva-de-agentes-de-ia-2/> (accessed Jun. 03, 2026).